DSC 40B - Homework 06

Due: Tuesday, November 10

Instructions:

- Write your solutions to the following problems **by hand**, either on paper that you scan or using a tablet. Typed solutions will not be accepted for credit!
  - Code is an exception. You do not need to handwrite code; you can instead include it (and any plots it makes) as a screenshot.
- Unless otherwise noted by the problem's instructions, show your work or provide some justification for your answer.
- Homework problems are graded pass/fail on effort, not correctness.
- Homeworks are due via Gradescope at 11:59 PM.

Problem 1. ($\frac{1}{2}$ credit)

In the following, let

$$V = \{0, 1, 2, 3, 4, 5, 6, 7\},$$
$$E = \{(0, 1), (1, 5), (5, 2), (0, 5), (7, 3), (6, 7)\}.$$

For this problem, you do not need to show your work.

a) Draw the *undirected* graph $G = (V, E)$. Remember that when writing the edges of an undirected graph, we often abuse notation and write (u, v) when we really mean $\{u, v\}$; we have done so here.

b) Draw the graph $G = (V, E)$, assuming that G is directed.

c) Write down the connected components of G, assuming that G is undirected.

d) Write the adjacency matrix representation of G, assuming that G is undirected.

e) Write the adjacency matrix representation of G, assuming that G is directed.

Problem 2. (1 credit)

Suppose A is the adjacency matrix of an undirected graph. Let A^2 be the *squared* matrix, obtained by matrix multiplying A by itself. Show that the (i, j) entry of A^2 is the number of ways to get from i to j in exactly two hops (i.e., the number of paths of length two between node i and node j). *Hint:* Consult your linear algebra notes/textbook to remember a formula for the (i, j) entry of the product of two matrices.

Problem 3. ($\frac{1}{2}$ credit)

Consider the following modified full BFS, which has two new lines of code: lines 21 and 22. Assume that the graph is undirected.

```
1  from collections import deque
2
3  def foo(graph):
```

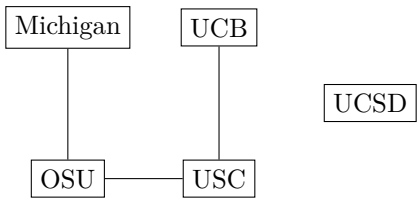
```
4       status = {node: 'undiscovered' for node in graph.nodes}
5       for u in graph.nodes:
6           if status[u] == 'undiscovered':
7               bar(graph, u, status)
8
9   def bar(graph, source, status):
10      status[source] = 'pending'
11      pending = deque([source])
12
13      # while there are still pending nodes
14      while pending:
15          u = pending.popleft()
16          for v in graph.neighbors(u):
17              # explore edge (u,v)
18              if status[v] == 'undiscovered':
19                  status[v] = 'pending'
20                  pending.append(v)
21              for x in graph.nodes:
22                  print(x)
23          status[u] = 'visited'
```

a) In total, over the whole run of `foo`, how many times is line 22 (`print(x)`) executed? Give your answer in terms of $|V|$ and $|E|$, and briefly explain.

b) What is the time complexity of `foo` in terms of $|V|$ and $|E|$? Justify your answer.

Programming Problem 1. (1 credit)

We can use a graph to represent rivalries between universities. Each node in the graph is a university, and an edge exists between two nodes if those two schools are rivals. For instance, the graph below represents the fact that OSU and Michigan are rivals, OSU and USC are rivals, UCB and USC are rivals, but UCSD does not have a rival.



In a file called `assign_good_and_evil.py`, write a function `assign_good_and_evil(graph)` which determines if it is possible to label each university as either "good" or "evil" such that every rivalry is between a "good" school and an "evil" school. The input to the function will be a graph of type `UndirectedGraph` from the `dsc40graph` package. If there is a way to label each node as "good" or "evil" so that every rivalry is between a "good" school and an "evil" school, your function should return it as a dictionary mapping each node to a string, `'good'` or `'evil'`; if such a labeling is not possible, your function should return `None`.

For example:

```
>>> example_graph = dsc40graph.UndirectedGraph()
>>> example_graph.add_edge('Michigan', 'OSU')
>>> example_graph.add_edge('USC', 'OSU')
>>> example_graph.add_edge('USC', 'UCB')
>>> example_graph.add_node('UCSD')
>>> assign_good_and_evil(example_graph)
{
    'OSU': 'good',
```

```
    'Michigan': 'evil',
    'USC': 'evil',
    'UCB': 'good',
    'UCSD': 'good'
}
```

If the above graph also had an edge between `'Michigan'` and `'USC'`, no such labeling would exist, and your function should return `None`.

Of course, there might be several ways to label the graph – your code need only return one labeling.

Your function should take $O(|V|+|E|)$ time. The autograder will check this by running your code on a large graph.

You can install `dsc40graph` with `pip install dsc40graph`, as on the "Dict-of-sets implementation" slide in Lecture 11. Its documentation is at `https://eldridgejm.github.io/dsc40graph/`.

As this is a programming problem, submit your `assign_good_and_evil.py` file to the Gradescope autograder assignment named "Homework 06 - Programming Problem 01", which has the same due date as the written homework. You don't need to write your code by hand or include it (or a screenshot of it) in your written homework; submitting it to the autograder is enough.